MIT SCHOOL OF COMPUTING
Rajbaug, Loni-Kalbhor, Pune

MIT-ADT UNIVERSITY
PUNE, INDIA
A Leap Towards World Class Education

Name: Abdulhady Salim Nawar Class: SY-5 Batch: A
Subject: PPL Roll No.: 03 Exp. No.: 1
Name of the Experiment: Decorator

Performed on:
Submitted on:

Marks	Teacher's Signature with date
10	

Experiment 1

Q.1 What is a Python decorator?

→ A python decorator is a function that takes another function as input, adds extra behaviour to it and returns a new modified function. It allows us to extend functionality without modifying the original function's code. Example uses:

- Logging
- Authentication
- Timing execution
- Caching.

Q.2 Why use a decorator for logging execution time?

→ We use a decorator because it avoids repeating ~~the~~ timing ~~same~~ code inside every function. It separates logic from monitoring logic and keeps the original function clean.

Incomplete for Theory ☐ Diagrams ☐ Observation Table ☐
Calculations ☐ Graphs ☐ Results ☐ Conclusion ☐
Understanding Through Q/A ☐ Late Submission ☐ Neatness ☐

Teacher's Signature

without decorator we'd have to write this in every function:

```
start= time.time()
end= time.time()
```

with decorator just write:

```
@log_time
def my_func():
```

Q.3 ~~Why do we use~~ How does time.time() help in measuring execution time?

→ time.time() returns the current time in seconds.

To measure execution time:

```
start= time.time()
func()
end= time.time()
execution_time= end - start
```

~~The~~ Execution_time returns seconds with decimal precision.

Q.4 Why do we use functools.wraps(func) inside the decorator?

→ When we use a decorator on a function, the original function gets replaced by the wrapper function. Without 'wraps', Python thinks the function's name is 'wrapper' instead of the original function's name. Therefore, functools.wraps(func)

- Preserves the original function's name
- Preserves metadata
- makes debugging easier.

Without wraps:

~~my_function.name~~ my_function.__name__ → wrapper.

Q.5 Can we use this decorator for any function?

→ If the wrapper uses:

```
def wrapper(*args, **kwargs):
```

Then it can handle any number of positional arguments and any number of keyword arguments. This makes the decorator reusable for almost any function.

Experiment 1-Write a Python program to create a decorator that counts the number of times a function is called.

Code-

```python
from functools import wraps

def count_calls(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        wrapper.call_count += 1
        print(f"Function '{func.__name__}' has been called {wrapper.call_count} times")
        return func(*args, **kwargs)

    wrapper.call_count = 0
    return wrapper


@count_calls
def greet(name):
    print(f"Hello {name}")


# Testing
greet("Abdul")
greet("Ziyan")
greet("Mitansh")

print(f"Total calls: {greet.call_count}")
```

Output-

```
PS C:\Users\Microsoft\OneDrive\Desktop\abdulfile> & "C:/Program Files/Python314/python.exe" c:/Users/Mic
rosoft/OneDrive/Desktop/abdulfile/assignment.py
Function 'greet' has been called 1 times
Hello Abdul
Function 'greet' has been called 2 times
Hello Ziyan
Function 'greet' has been called 3 times
Hello Mitansh
Total calls: 3
PS C:\Users\Microsoft\OneDrive\Desktop\abdulfile>
```